

Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 18

Dictionaries and methods



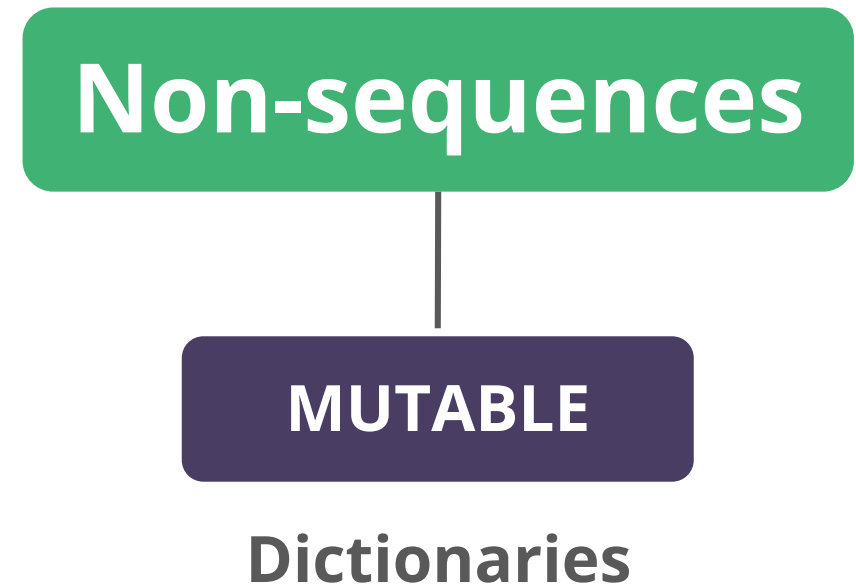
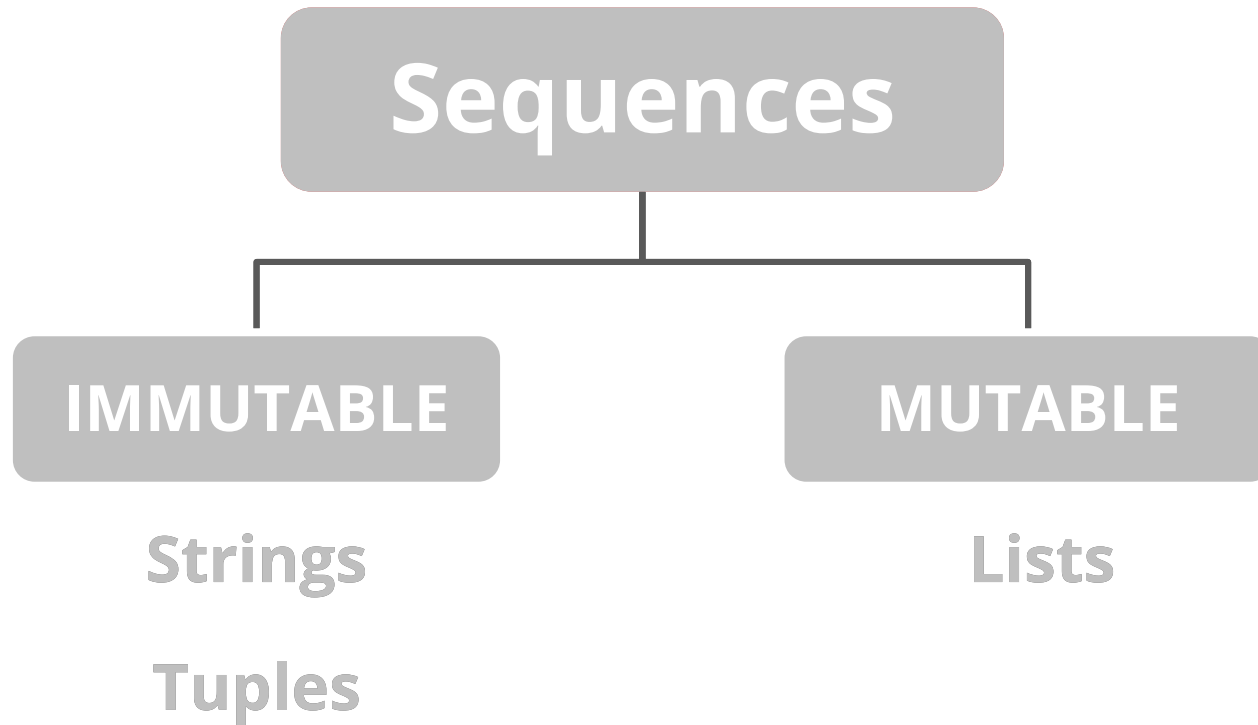
Università
Bocconi
MILANO

Objectives of the lesson

Learning how to access and handle dictionaries



Sequences -vs- Non-sequences



Dictionaries

- In Python, a **dictionary** is an object that contains a collection of elements
- Each element consists of two parts: a **key** and a **value**. The keys are unique and can be any kind of immutable objects. Values can be any kind of object, mutable or immutable
- To create a dictionary, it is necessary to write its elements separated by commas (,) and enclosed within **curly brackets** {}. Each element contains a key and a value separated by a colon (:)
- Alternatively, you can use the **dict** function to create an empty dictionary
- Dictionaries are **mutable** objects

Keys and values

- The order of dictionary's elements is unpredictable, therefore we cannot use indexing
- To retrieve a value stored in the dictionary, we must use the corresponding key :

`dict_name[key]`

- In order to add a new element to a dictionary, an assignment must be used:

`dict_name[key] = value`

Operations with dictionaries

- **in** allows to check if a **key** is contained in a dictionary
- **del** removes an element (i.e. a key-value pair) from a dictionary
- **len** returns the number of elements (i.e. the key-value pairs) of a dictionary

Methods of dictionaries

Method	Description
<code>.get(key[,default])</code>	Returns the value corresponding to the key specified by the <i>key</i> argument. If <i>key</i> is not found, it returns <i>None</i> or the value specified in the <i>default</i> argument.
<code>.pop(key[,default])</code>	Removes the key-value pair corresponding to the <i>key</i> argument and returns its value. If <i>key</i> is not found in the dictionary and a default value has not been set in the <i>default</i> argument, it returns a <i>KeyError</i> error.
<code>.popitem()</code>	Removes the last key-value pair added to the dictionary and returns a tuple with two elements, i.e. the key and the value removed.
<code>.items()</code>	Creates a <i>dict_items</i> object consisting in a list of tuples representing the key-value pairs (i.e. each tuple contains a key and the corresponding value).
<code>.keys()</code>	Creates a <i>dict_keys</i> object consisting in a list which contains the keys of the dictionary.
<code>.values()</code>	Creates a <i>dict_values</i> object consisting in a list which contains the values of the dictionary.



Traversing dictionaries

- The keys of a dictionary can be traversed using a **for** loop:

```
address_book = { 'Julia': '555-123456', 'Mark': '555-654321' }  
for K in address_book:  
    print(K)
```

- We can also traverse key-value pairs using a **for** loop and the **.items()** method:

```
for K, Val in address_book.items():  
    print(K, Val)
```



Exercise 1

Create a new Python file, named **Exercise 1.py**, containing a function named **create** which creates a dictionary. In particular, the function must:

- have an integer as a mandatory parameter
- create a dictionary in which the keys are integer numbers between 1 and the integer value passed to the function as a mandatory argument, while values are calculated using the formula $(\text{key}/2)^3$
- return the dictionary

Create now a second function, named **show**, to show all the keys and the corresponding values of a dictionary. The function must:

- have a dictionary as a mandatory parameter
- show on screen the keys and the corresponding values, separated by a tabulation character

Exercise 2

In a new Python file, named **Exercise 2.py**, create the function **delete** to delete some key-value pairs from a dictionary, based on the keys present in a sequence. In particular, the function must:

- have a dictionary, from which to delete the items, and a sequence of keys as mandatory parameters
- delete from the dictionary all the items (i.e. key-value pairs) corresponding to the keys contained in the list passed as an argument
- show a message on screen specifying the deleted key-value pairs
- return the dictionary's final version (after the elements removal)

To test the function, it is possible to use variable **a1**:

```
a1 = {'a':61, 'b':22, 'c':53, 'd':19, 'e':54, 'f':34, 'z':71, 'h':26, 'l':3, 'p':64, 'y':5, 'u':54}
```

Exercise 3

In a new Python file, named **Exercise 3.py**, create the function **frequency** to calculate the number of occurrences of characters in a text string. In particular, the function must:

- have a text string as a mandatory parameter
- create a dictionary in which the keys are the string's characters, and the values are the corresponding number of occurrences
- return the dictionary

Tools to learn with today's lecture

- How to handle and use dictionaries

Book references

Understanding Python:

Chapters 9

Python workbook:

Exercises of Unit 10